

Phase 2: Binary Tree Mathematics and Counting Shapes

0. Purpose of This Phase

Phase 2 teaches the mathematics of ordinary binary trees.

In Phase 1, you learned what a tree is and what a binary tree is. Now you will learn how to answer questions like:

- If the height is known, how many nodes can the tree have?
- If the number of nodes is known, how tall can the tree be?
- How are leaf nodes connected to degree-2 nodes?
- How many different binary tree shapes are possible?
- What is the difference between unlabeled and labeled binary trees?

The main beginner idea is:

A binary tree has shape limits. We can use formulas to count those limits.

1. Current Progress Analysis

Before Phase 2, you should already know these Phase 1 ideas:

- A tree is made of nodes and edges.
- A binary tree is a tree where each node has at most two children.
- The two children are called left child and right child.
- A leaf node has no children.
- An internal node has at least one child.
- The degree of a node is the number of direct children it has.
- Height counts edges from the root down to the deepest node.
- In this curriculum, a single root has height 0.

So your current position is:

You know what a binary tree is. Now you are learning the formulas and counting ideas behind binary tree shapes.

2. Important Height Convention

In this curriculum, height starts from 0.

2.1 Single Node Tree

A

This tree has:

number of nodes = 1
height = 0

Why?

Because height counts edges, and there are no edges below the root.

2.2 Tree with One Edge

A

B

This tree has one edge:

A -> B

So:

height = 1

2.3 Tree with Two Edges

A

B

C

The longest path is:

A -> B -> C

There are 2 edges.

So:

height = 2

3. Height vs Number of Nodes

This section answers:

If I know the height of a binary tree, what is the minimum or maximum number of nodes it can have?

There are two important cases:

1. Minimum nodes for a given height
2. Maximum nodes for a given height

4. Minimum Nodes for Height H

To get height **H** using the fewest nodes, make the tree like a chain.

This kind of tree is also called a skewed tree.

4.1 Example: Height 3

A
 B
 C
 D

Count the nodes:

A, B, C, D = 4 nodes

Count the edges on the longest path:

A -> B = 1
B -> C = 2
C -> D = 3

So:

height = 3
nodes = 4

4.2 Formula

Minimum nodes for height $H = H + 1$

4.3 Example Calculation

If:

$H = 3$

Then:

Minimum nodes = $H + 1$
 = $3 + 1$
 = 4

4.4 Simple Meaning

For minimum nodes, think:

One node per level

A height of 3 means there are 4 node positions on the longest path:

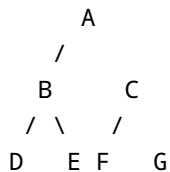
height 0, height 1, height 2, height 3

So the minimum number of nodes is 4.

5. Maximum Nodes for Height H

To get the maximum number of nodes for height H , fill every level completely.

5.1 Example: Height 2



Now count level by level:

height 0: 1 node -> A
height 1: 2 nodes -> B, C
height 2: 4 nodes -> D, E, F, G

Total:

$$1 + 2 + 4 = 7$$

So for height 2, the maximum number of nodes is 7.

5.2 Formula

Maximum nodes for height $H = 2^{(H + 1)} - 1$

5.3 Example Calculation

If:

$$H = 2$$

Then:

$$\begin{aligned}\text{Maximum nodes} &= 2^{(H + 1)} - 1 \\ &= 2^{(2 + 1)} - 1 \\ &= 2^3 - 1 \\ &= 8 - 1 \\ &= 7\end{aligned}$$

5.4 Why the Formula Uses $H + 1$

Height starts at 0.

So if height is 2, the levels are:

```
height 0
height 1
height 2
```

That is 3 levels total.

So the number of levels is:

$$H + 1$$

That is why the formula is:

$$2^{(H + 1)} - 1$$

not:

$$2^H - 1$$

6. Node Range for a Given Height

For a binary tree with height H :

$$\begin{aligned}\text{Minimum nodes} &= H + 1 \\ \text{Maximum nodes} &= 2^{(H + 1)} - 1\end{aligned}$$

So the number of nodes must be between these two values.

6.1 Table

Height H	Minimum Nodes H + 1	Maximum Nodes $2^{(H + 1)} - 1$
0	1	1
1	2	3
2	3	7
3	4	15
4	5	31

6.2 Simple Memory Trick

Minimum nodes = skinny tree
Maximum nodes = fully packed tree

7. Height from Number of Nodes

Now we reverse the question.

Instead of asking:

Given height, how many nodes?

We ask:

Given nodes, what height?

There are two important cases:

1. Minimum height for a given number of nodes
2. Maximum height for a given number of nodes

8. Maximum Height from N Nodes

To make height as large as possible, make the tree a chain.

8.1 Example: 5 Nodes

A
B
C
D
E

There are 5 nodes:

A, B, C, D, E

The longest path has 4 edges:

A -> B = 1
B -> C = 2
C -> D = 3
D -> E = 4

So:

maximum height = 4

8.2 Formula

Maximum height from N nodes = $N - 1$

8.3 Example Calculation

If:

$N = 5$

Then:


```
Maximum height = N - 1  
                = 5 - 1  
                = 4
```

8.4 Simple Meaning

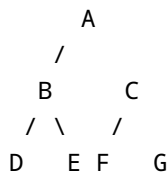
For maximum height, think:

Make the tree as long and thin as possible.

9. Minimum Height from N Nodes

To make height as small as possible, pack the nodes as tightly as possible.

9.1 Example: 7 Nodes



There are 7 nodes.

The height is 2.

So:

```
N = 7  
minimum height = 2
```

9.2 Formula

```
Minimum height from N nodes =  $\text{ceil}(\log_2(N + 1)) - 1$ 
```

9.3 Example Calculation: N = 7

```
Minimum height =  $\text{ceil}(\log_2(N + 1)) - 1$   
                =  $\text{ceil}(\log_2(7 + 1)) - 1$ 
```

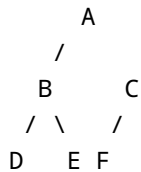
$$\begin{aligned}
 &= \text{ceil}(\log_2(8)) - 1 \\
 &= \text{ceil}(3) - 1 \\
 &= 3 - 1 \\
 &= 2
 \end{aligned}$$

So 7 nodes can be packed into height 2.

9.4 Why We Use Ceiling

Sometimes the number of nodes does not perfectly fill the tree.

Example with 6 nodes:



This tree has:

$$\begin{aligned}
 N &= 6 \\
 \text{height} &= 2
 \end{aligned}$$

Now calculate:

$$\begin{aligned}
 \text{Minimum height} &= \text{ceil}(\log_2(6 + 1)) - 1 \\
 &= \text{ceil}(\log_2(7)) - 1
 \end{aligned}$$

Since:

$$\log_2(7) \text{ is about } 2.8$$

Then:

$$\text{ceil}(2.8) = 3$$

So:

$$\begin{aligned}\text{Minimum height} &= 3 - 1 \\ &= 2\end{aligned}$$

We use `ceil` because partial levels still need a full height level.

10. Height Range for a Given Number of Nodes

For a binary tree with `N` nodes:

$$\begin{aligned}\text{Minimum height} &= \text{ceil}(\log_2(N + 1)) - 1 \\ \text{Maximum height} &= N - 1\end{aligned}$$

10.1 Table

Nodes N	Minimum Height	Maximum Height
1	0	0
2	1	1
3	1	2
4	2	3
5	2	4
6	2	5
7	2	6

10.2 Simple Memory Trick

Minimum height = packed tightly
Maximum height = long chain

11. Degree 0, Degree 1, and Degree 2 Nodes

In a binary tree, each node can have at most two children.

So a node can have:

```
0 children
1 child
2 children
```

These are called degree 0, degree 1, and degree 2 nodes.

12. Degree 0 Node

A degree 0 node has no children.

A degree 0 node is also called:

```
leaf node
external node
terminal node
```

12.1 Example

```
A
```

Here, **A** has no children.

So:

```
degree(A) = 0
```

A is a leaf node.

13. Degree 1 Node

A degree 1 node has exactly one child.

13.1 Example

```
A
  B
```

Here:

A has one child: B

So:

$\text{degree}(A) = 1$

14. Degree 2 Node

A degree 2 node has exactly two children.

14.1 Example

A
/
B C

Here:

A has two children: B and C

So:

$\text{degree}(A) = 2$

15. Degree Type Summary

Node Type	Number of Children	Simple Meaning
Degree 0	0	Leaf node
Degree 1	1	Node with one child
Degree 2	2	Node with two children

16. Important Formula: Degree 0 and Degree 2 Relationship

For every non-empty binary tree:

$$\text{degree}(0) = \text{degree}(2) + 1$$

This means:

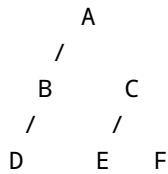
$$\text{Number of leaf nodes} = \text{Number of degree-2 nodes} + 1$$

Or:

$$\text{Leaves} = \text{degree-2 nodes} + 1$$

17. Example of Degree Formula

Consider this tree:



Now count the degree of each node.

17.1 Count Each Node

A has 2 children -> degree 2
B has 1 child -> degree 1
C has 2 children -> degree 2
D has 0 children -> degree 0
E has 0 children -> degree 0
F has 0 children -> degree 0

So:

degree 0 nodes = 3 -> D, E, F
degree 1 nodes = 1 -> B
degree 2 nodes = 2 -> A, C

17.2 Check the Formula

Formula:

$$\text{degree}(0) = \text{degree}(2) + 1$$

Substitute values:

$$3 = 2 + 1$$

So the formula is correct.

18. Important Note About Degree 1 Nodes

Degree 1 nodes do not affect the formula.

The formula is only about:

degree 0 nodes
and
degree 2 nodes

Even if the tree has many degree 1 nodes, the formula still works.

18.1 Example: Chain Tree

A
 B
 C
 D

Degrees:

```
A has 1 child -> degree 1
B has 1 child -> degree 1
C has 1 child -> degree 1
D has 0 children -> degree 0
```

So:

```
degree 0 nodes = 1
degree 2 nodes = 0
```

Check:

```
degree(0) = degree(2) + 1
1 = 0 + 1
```

Correct.

19. Counting Binary Tree Shapes

Now we ask a new question:

How many different binary tree shapes can be made with nodes?

This is not asking about height.

This is asking about shape.

20. What Does Shape Mean?

Shape means the structure of the tree.

It means:

- Which nodes have left children
- Which nodes have right children
- How the tree branches

Shape does not care about the actual data values stored inside the nodes.

21. Unlabeled Binary Trees

An unlabeled binary tree only cares about shape.

The nodes do not have important names or values.

Example:



Here, we only care that:

one root has two children

We do not care if the root stores `A`, `10`, or `John`.

22. Catalan Numbers

The number of different binary tree shapes with `n` nodes is the n th Catalan number.

Formula:

$$\text{Catalan}(n) = C(2n, n) / (n + 1)$$

Here:

$$C(2n, n)$$

means combination:

choose n items from $2n$ items

23. Catalan Example: $n = 3$

Formula:

$$\text{Catalan}(n) = C(2n, n) / (n + 1)$$

For:

$$n = 3$$

We get:

$$\text{Catalan}(3) = C(6, 3) / 4$$

Now:

$$C(6, 3) = 20$$

So:

$$\begin{aligned}\text{Catalan}(3) &= 20 / 4 \\ &= 5\end{aligned}$$

Therefore:

3 nodes can form 5 different binary tree shapes.

24. The 5 Binary Tree Shapes with 3 Nodes

Shape 1

```
  A
 /
B
 /
C
```

Shape 2

A
/
B

C

Shape 3

A
/
B C

Shape 4

A

B
/
C

Shape 5

A

B

C

So:

Catalan(3) = 5

25. Catalan Example: $n = 5$

Formula:

$$\text{Catalan}(n) = C(2n, n) / (n + 1)$$

For:

$$n = 5$$

We get:

$$\text{Catalan}(5) = C(10, 5) / 6$$

Now:

$$C(10, 5) = 252$$

So:

$$\begin{aligned}\text{Catalan}(5) &= 252 / 6 \\ &= 42\end{aligned}$$

Therefore:

5 nodes can form 42 different binary tree shapes.

26. Small Catalan Number Table

n	Catalan(n)	Meaning
0	1	Empty tree shape
1	1	One-node tree shape
2	2	Two possible shapes
3	5	Five possible shapes
4	14	Fourteen possible shapes
5	42	Forty-two possible shapes
6	132	One hundred thirty-two possible shapes

27. Why Catalan(0) = 1

This may look strange at first.

Catalan(0) = 1

This means:

There is one empty tree possibility.

In recursion, an empty left subtree or empty right subtree is still counted as one valid possibility.

That is why:

$T_0 = 1$

28. Recursive Catalan Idea

The recursive Catalan idea is:

Choose one node as the root.
Then split the remaining nodes between the left subtree and the right subtree.

For every possible split:

number of left shapes * number of right shapes

Then add all the possibilities.

29. Recursive Catalan Formula

Let:

T_n = number of binary tree shapes with n nodes

Base values:

$T_0 = 1$
 $T_1 = 1$

Formula:

$T_n = T_0 * T(n-1) + T_1 * T(n-2) + T_2 * T(n-3) + \dots + T(n-1) * T_0$

Another way to write it:

$T_n = \text{sum of } T(\text{left nodes}) * T(\text{right nodes})$

where:

$\text{left nodes} + \text{right nodes} = n - 1$

Why $n - 1$?

Because one node is used as the root.

30. Recursive Catalan Dry Run: T3

We want:

T_3

This means:

number of binary tree shapes using 3 nodes

One node is the root.

So remaining nodes:

$3 - 1 = 2$

Possible splits:

```
left 0, right 2
left 1, right 1
left 2, right 0
```

So:

$$T_3 = T_0 * T_2 + T_1 * T_1 + T_2 * T_0$$

Known values:

```
T0 = 1
T1 = 1
T2 = 2
```

Substitute:

$$\begin{aligned} T_3 &= 1 * 2 + 1 * 1 + 2 * 1 \\ &= 2 + 1 + 2 \\ &= 5 \end{aligned}$$

So:

$$T_3 = 5$$

31. Recursive Catalan Dry Run: T6

Known values:

```
T0 = 1
T1 = 1
T2 = 2
T3 = 5
T4 = 14
T5 = 42
```

Formula:

$$T_6 = T_0 \cdot T_5 + T_1 \cdot T_4 + T_2 \cdot T_3 + T_3 \cdot T_2 + T_4 \cdot T_1 + T_5 \cdot T_0$$

Substitute:

$$T_6 = 1 \cdot 42 + 1 \cdot 14 + 2 \cdot 5 + 5 \cdot 2 + 14 \cdot 1 + 42 \cdot 1$$

Calculate:

$$\begin{aligned} T_6 &= 42 + 14 + 10 + 10 + 14 + 42 \\ &= 132 \end{aligned}$$

So:

$$T_6 = 132$$

32. Maximum-Height Binary Trees

Maximum height means the tree is as tall as possible.

That means the tree is a chain.

32.1 Example with 4 Nodes

```
A
  B
    C
      D
```

This tree has:

$$\begin{aligned} N &= 4 \\ \text{height} &= 3 \end{aligned}$$

This is maximum height because:


```
Maximum height = N - 1  
                = 4 - 1  
                = 3
```

33. Counting Maximum-Height Trees

In a maximum-height tree, every node except the last has exactly one child.

At each step, the next child can go:

```
left  
or  
right
```

For nodes, there are:

```
n - 1
```

connections below the root.

Each connection has 2 choices:

```
left or right
```

So the number of maximum-height binary trees is:

```
2^(n - 1)
```

34. Maximum-Height Tree Example: n = 4

Formula:

```
Number of maximum-height trees = 2^(n - 1)
```

For:

$$n = 4$$

Then:

$$\begin{aligned}\text{Number} &= 2^{(4 - 1)} \\ &= 2^3 \\ &= 8\end{aligned}$$

So:

There are 8 maximum-height binary trees with 4 nodes.

35. Maximum-Height Tree Example: $n = 5$

Formula:

$$\text{Number of maximum-height trees} = 2^{(n - 1)}$$

For:

$$n = 5$$

Then:

$$\begin{aligned}\text{Number} &= 2^{(5 - 1)} \\ &= 2^4 \\ &= 16\end{aligned}$$

So:

There are 16 maximum-height binary trees with 5 nodes.

36. Labeled and Unlabeled Binary Trees

This is an important difference.

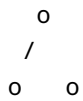
There are two ways to count binary trees:

1. Unlabeled counting
2. Labeled counting

37. Unlabeled Binary Trees

In an unlabeled binary tree, only shape matters.

Example:



We do not care what values are inside the nodes.

So for unlabeled binary trees:

Number of trees = Catalan(n)

38. Labeled Binary Trees

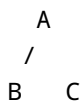
In a labeled binary tree, the node values matter.

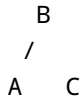
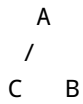
Example labels:

A, B, C

The same shape can have many different label arrangements.

38.1 Same Shape, Different Labels





These have the same shape, but the labels are arranged differently.

So they are different labeled trees.

39. Labeled Binary Tree Formula

For each shape, the labels can be arranged in:

$n!$

ways.

So:

Number of labeled binary trees = Catalan(n) * $n!$

40. Labeled Binary Tree Example: $n = 3$

First find the number of shapes:

Catalan(3) = 5

Then find label arrangements:

$3! = 3 * 2 * 1 = 6$

Now multiply:

```
Labeled binary trees = Catalan(3) * 3!  
                     = 5 * 6  
                     = 30
```

So:

There are 30 labeled binary trees with 3 nodes.

41. Unlabeled vs Labeled Summary

Type	What Matters?	Formula
Unlabeled binary tree	Shape only	Catalan(n)
Labeled binary tree	Shape + label arrangement	Catalan(n) * n!

41.1 Beginner Memory Trick

```
Unlabeled = shape only  
Labeled = shape + names/values
```

42. Complete Formula Summary

Question	Formula
Minimum nodes for height H	$H + 1$
Maximum nodes for height H	$2^{(H + 1)} - 1$
Minimum height for N nodes	$\text{ceil}(\log_2(N + 1)) - 1$
Maximum height for N nodes	$N - 1$
Leaf and degree-2 relation	$\text{degree}(0) = \text{degree}(2) + 1$
Unlabeled binary tree shapes	Catalan(n)
Catalan formula	$C(2n, n) / (n + 1)$
Recursive Catalan formula	$T_n = T_0T_{(n-1)} + T_1T_{(n-2)} + \dots + T_{(n-1)}T_0$
Maximum-height binary trees	$2^{(n - 1)}$
Labeled binary trees	Catalan(n) * n!

43. Formula Meaning Table

Formula	Simple Meaning
$H + 1$	Fewest nodes needed to create height H
$2^{(H + 1)} - 1$	Most nodes possible for height H
$N - 1$	Tallest possible height using N nodes
$\text{ceil}(\log_2(N + 1)) - 1$	Shortest possible height using N nodes
$\text{degree}(0) = \text{degree}(2) + 1$	Leaves are always one more than degree-2 nodes
$\text{Catalan}(n)$	Number of unlabeled shapes
$\text{Catalan}(n) * n!$	Number of labeled binary trees
$2^{(n - 1)}$	Number of chain-shaped maximum-height trees

44. C++ Code: Height and Node Formulas

```
#include <iostream>
#include <cmath>
using namespace std;

int minNodesFromHeight(int h) {
    return h + 1;
}

int maxNodesFromHeight(int h) {
    return (int)pow(2, h + 1) - 1;
}

int maxHeightFromNodes(int n) {
    return n - 1;
}

int minHeightFromNodes(int n) {
    return (int)ceil(log2(n + 1)) - 1;
}

int main() {
    int h = 3;
    int n = 15;
```

```

    cout << "For height " << h << endl;
    cout << "Minimum nodes = " << minNodesFromHeight(h) << endl;
    cout << "Maximum nodes = " << maxNodesFromHeight(h) << endl;

    cout << "\nFor nodes " << n << endl;
    cout << "Minimum height = " << minHeightFromNodes(n) << endl;
    cout << "Maximum height = " << maxHeightFromNodes(n) << endl;

    return 0;
}

```

44.1 Expected Output

```

For height 3
Minimum nodes = 4
Maximum nodes = 15

For nodes 15
Minimum height = 3
Maximum height = 14

```

45. C++ Code: Counting Degree 0, Degree 1, and Degree 2 Nodes

```

#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};

void countByDegree(Node* root, int& deg0, int& deg1, int& deg2) {
    if (root == nullptr) {
        return;
    }
}

```

```

bool hasLeft = root->left != nullptr;
bool hasRight = root->right != nullptr;

if (!hasLeft && !hasRight) {
    deg0++;
}
else if (hasLeft && hasRight) {
    deg2++;
}
else {
    deg1++;
}

countByDegree(root->left, deg0, deg1, deg2);
countByDegree(root->right, deg0, deg1, deg2);
}

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->right->left = new Node(5);
    root->right->right = new Node(6);

    int deg0 = 0;
    int deg1 = 0;
    int deg2 = 0;

    countByDegree(root, deg0, deg1, deg2);

    cout << "Degree 0 nodes: " << deg0 << endl;
    cout << "Degree 1 nodes: " << deg1 << endl;
    cout << "Degree 2 nodes: " << deg2 << endl;
    cout << "Formula check: degree(0) = degree(2) + 1" << endl;
    cout << deg0 << " = " << deg2 << " + 1" << endl;

    return 0;
}

```

45.1 Tree Created by the Code

```

    1
   /
  2   3

```


/ /
4 5 6

45.2 Degree Count

Degree 0 nodes: 4, 5, 6 -> 3 nodes
Degree 1 nodes: 2 -> 1 node
Degree 2 nodes: 1, 3 -> 2 nodes

Formula check:

degree(0) = degree(2) + 1
3 = 2 + 1

46. C++ Code: Catalan Number by Combination Formula

```
#include <iostream>
using namespace std;

long long catalanCombination(int n) {
    long long combination = 1;

    // Calculate C(2n, n)
    for (int i = 0; i < n; i++) {
        combination = combination * (2 * n - i);
        combination = combination / (i + 1);
    }

    // Catalan(n) = C(2n, n) / (n + 1)
    return combination / (n + 1);
}

int main() {
    int n = 5;

    cout << "Catalan(" << n << ") = " << catalanCombination(n) << endl;

    return 0;
}
```

46.1 Expected Output

```
Catalan(5) = 42
```

47. C++ Code: Catalan Number by Dynamic Programming

```
#include <iostream>
#include <vector>
using namespace std;

long long catalanDP(int n) {
    vector<long long> T(n + 1, 0);

    T[0] = 1;

    if (n >= 1) {
        T[1] = 1;
    }

    for (int i = 2; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            T[i] += T[j - 1] * T[i - j];
        }
    }

    return T[n];
}

int main() {
    cout << "Catalan(3) = " << catalanDP(3) << endl;
    cout << "Catalan(5) = " << catalanDP(5) << endl;
    cout << "Catalan(6) = " << catalanDP(6) << endl;

    return 0;
}
```

47.1 Expected Output

```
Catalan(3) = 5
Catalan(5) = 42
Catalan(6) = 132
```

48. C++ Code: Labeled Binary Tree Count

```
#include <iostream>
using namespace std;

long long catalanCombination(int n) {
    long long combination = 1;

    for (int i = 0; i < n; i++) {
        combination = combination * (2 * n - i);
        combination = combination / (i + 1);
    }

    return combination / (n + 1);
}

long long factorial(int n) {
    long long result = 1;

    for (int i = 1; i <= n; i++) {
        result *= i;
    }

    return result;
}

int main() {
    int n = 3;

    long long shapes = catalanCombination(n);
    long long arrangements = factorial(n);
    long long labeledTrees = shapes * arrangements;

    cout << "Unlabeled shapes = " << shapes << endl;
    cout << "Label arrangements = " << arrangements << endl;
    cout << "Labeled binary trees = " << labeledTrees << endl;

    return 0;
}
```

48.1 Expected Output

```
Unlabeled shapes = 5
Label arrangements = 6
Labeled binary trees = 30
```

49. Complete Phase 2 Example Program

This program combines the main Phase 2 ideas:

- Minimum and maximum nodes from height
- Minimum and maximum height from nodes
- Counting degree 0, degree 1, and degree 2 nodes
- Catalan number
- Labeled binary tree count

```
#include <iostream>
#include <cmath>
#include <vector>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};

int minNodesFromHeight(int h) {
    return h + 1;
}

int maxNodesFromHeight(int h) {
    return (int)pow(2, h + 1) - 1;
}

int maxHeightFromNodes(int n) {
    return n - 1;
}
```

```

int minHeightFromNodes(int n) {
    return (int)ceil(log2(n + 1)) - 1;
}

void countByDegree(Node* root, int& deg0, int& deg1, int& deg2) {
    if (root == nullptr) {
        return;
    }

    bool hasLeft = root->left != nullptr;
    bool hasRight = root->right != nullptr;

    if (!hasLeft && !hasRight) {
        deg0++;
    }
    else if (hasLeft && hasRight) {
        deg2++;
    }
    else {
        deg1++;
    }

    countByDegree(root->left, deg0, deg1, deg2);
    countByDegree(root->right, deg0, deg1, deg2);
}

long long catalanCombination(int n) {
    long long combination = 1;

    for (int i = 0; i < n; i++) {
        combination = combination * (2 * n - i);
        combination = combination / (i + 1);
    }

    return combination / (n + 1);
}

long long catalanDP(int n) {
    vector<long long> T(n + 1, 0);

    T[0] = 1;

    if (n >= 1) {
        T[1] = 1;
    }

    for (int i = 2; i <= n; i++) {

```

```

        for (int j = 1; j <= i; j++) {
            T[i] += T[j - 1] * T[i - j];
        }
    }

    return T[n];
}

long long factorial(int n) {
    long long result = 1;

    for (int i = 1; i <= n; i++) {
        result *= i;
    }

    return result;
}

int main() {
    int h = 3;
    int n = 5;

    cout << "Height " << h << endl;
    cout << "Minimum nodes = " << minNodesFromHeight(h) << endl;
    cout << "Maximum nodes = " << maxNodesFromHeight(h) << endl;

    cout << "\nNodes 15" << endl;
    cout << "Minimum height = " << minHeightFromNodes(15) << endl;
    cout << "Maximum height = " << maxHeightFromNodes(15) << endl;

    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->right->left = new Node(5);
    root->right->right = new Node(6);

    int deg0 = 0;
    int deg1 = 0;
    int deg2 = 0;

    countByDegree(root, deg0, deg1, deg2);

    cout << "\nDegree counts" << endl;
    cout << "Degree 0 nodes = " << deg0 << endl;
    cout << "Degree 1 nodes = " << deg1 << endl;
    cout << "Degree 2 nodes = " << deg2 << endl;
    cout << "Formula check: " << deg0 << " = " << deg2 << " + 1" << endl;
}

```

```

        cout << "\nCatalan numbers" << endl;
        cout << "Unlabeled shapes for " << n << " nodes = " <<
catalanCombination(n) << endl;
        cout << "DP Catalan T6 = " << catalanDP(6) << endl;

        cout << "\nLabeled trees" << endl;
        cout << "Labeled trees for 3 nodes = " << catalanCombination(3) *
factorial(3) << endl;

        return 0;
    }

```

50. Dry Run of the Complete Program

50.1 Height 3

```

Minimum nodes = H + 1
               = 3 + 1
               = 4

```

```

Maximum nodes = 2^(H + 1) - 1
               = 2^4 - 1
               = 16 - 1
               = 15

```

So the program prints:

```

Minimum nodes = 4
Maximum nodes = 15

```

50.2 Nodes 15

```

Minimum height = ceil(log2(15 + 1)) - 1
                = ceil(log2(16)) - 1
                = ceil(4) - 1
                = 3

```

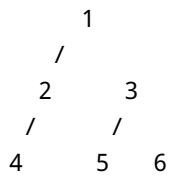
```
Maximum height = N - 1
                = 15 - 1
                = 14
```

So the program prints:

```
Minimum height = 3
Maximum height = 14
```

50.3 Degree Count Tree

The program creates this tree:



Count:

```
4, 5, 6 are leaves -> degree 0 = 3
2 has one child    -> degree 1 = 1
1 and 3 have two children -> degree 2 = 2
```

Formula check:

```
degree(0) = degree(2) + 1
3 = 2 + 1
```

Correct.

50.4 Catalan and Labeled Count

For 5 nodes:

```
Catalan(5) = 42
```


So:

5 nodes can form 42 unlabeled binary tree shapes.

For 3 labeled nodes:

Catalan(3) = 5
 $3! = 6$

So:

Labeled binary trees = $5 * 6 = 30$

51. Common Beginner Mistakes

Mistake 1: Using H Instead of H + 1

Wrong formula:

Maximum nodes = $2^H - 1$

Correct formula:

Maximum nodes = $2^{(H + 1)} - 1$

Why?

Because height starts at 0.

If height is 2, there are 3 levels:

0, 1, 2

So we use:

$H + 1$

Mistake 2: Forgetting Ceiling in Minimum Height

Wrong idea:

$$\text{Minimum height} = \log_2(N + 1) - 1$$

Correct idea:

$$\text{Minimum height} = \text{ceil}(\log_2(N + 1)) - 1$$

Why?

Because if the last level is not completely full, it still increases the height.

Example:

$$N = 6$$

The tree still needs height 2.

Mistake 3: Thinking Degree 1 Nodes Change the Leaf Formula

Wrong idea:

$$\text{Degree 1 nodes affect } \text{degree}(0) = \text{degree}(2) + 1$$

Correct idea:

Degree 1 nodes do not affect this formula.

The formula is always:

$$\text{degree}(0) = \text{degree}(2) + 1$$

for a non-empty binary tree.

Mistake 4: Mixing Up Labeled and Unlabeled Trees

Unlabeled means:

Only shape matters.

Labeled means:

Shape and node values both matter.

So:

Unlabeled binary trees = Catalan(n)
Labeled binary trees = Catalan(n) * n!

Mistake 5: Thinking Catalan Counts Only Full Trees

Catalan numbers count all possible binary tree shapes with n nodes.

They include:

- left-skewed shapes
- right-skewed shapes
- balanced shapes
- mixed shapes

Catalan does not only count full or complete trees.

Mistake 6: Forgetting Catalan(0) = 1

For recursive Catalan counting:

$T_0 = 1$

This means an empty subtree is one valid possibility.

Without this, the recursive formula will not work correctly.

52. Phase 2 Key Vocabulary Table

Term	Simple Meaning
Height	Number of edges on the longest path from root downward
Minimum nodes	Fewest nodes possible for a given height
Maximum nodes	Most nodes possible for a given height
Minimum height	Smallest height possible for a given number of nodes
Maximum height	Largest height possible for a given number of nodes
Degree 0 node	Node with no children; leaf node
Degree 1 node	Node with exactly one child
Degree 2 node	Node with exactly two children
Catalan number	Number of possible binary tree shapes with n nodes
Unlabeled tree	Tree where only shape matters
Labeled tree	Tree where node values/labels also matter
Factorial n!	Number of ways to arrange n labels
Maximum-height tree	A chain-like tree with height $N - 1$

53. Phase 2 Summary

By the end of Phase 2, you should know the following:

A single root has height 0.

Minimum nodes for height $H = H + 1$

Maximum nodes for height $H = 2^{(H + 1)} - 1$

Minimum height for N nodes = $\text{ceil}(\log_2(N + 1)) - 1$

Maximum height for N nodes = $N - 1$

A degree 0 node is a leaf.

A degree 1 node has exactly one child.

A degree 2 node has exactly two children.

$\text{degree}(0) = \text{degree}(2) + 1$

Catalan(n) counts unlabeled binary tree shapes.

$\text{Catalan}(n) = C(2n, n) / (n + 1)$

Recursive Catalan counts left-subtree and right-subtree possibilities.

Maximum-height binary trees = $2^{(n - 1)}$

Labeled binary trees = $\text{Catalan}(n) * n!$

54. Beginner Memory Tricks

Remember these short ideas:

Minimum nodes = skinny tree

Maximum nodes = fully packed tree

Maximum height = chain

Minimum height = packed tightly

Leaves = degree-2 nodes + 1

Catalan = number of shapes

Catalan * n! = number of labeled trees

55. Practice Questions

Question 1

If height $H = 3$, find:

minimum nodes
maximum nodes

Question 2

If height $H = 4$, find:

minimum nodes
maximum nodes

Question 3

If a binary tree has $N = 6$ nodes, find:

minimum height
maximum height

Question 4

If a binary tree has $N = 15$ nodes, find:

minimum height
maximum height

Question 5

If a binary tree has 4 degree-2 nodes, how many leaf nodes does it have?

Question 6

If a binary tree has 9 leaf nodes, how many degree-2 nodes does it have?

Question 7

How many unlabeled binary tree shapes are possible with 3 nodes?

Question 8

How many unlabeled binary tree shapes are possible with 5 nodes?

Question 9

How many maximum-height binary trees are possible with 5 nodes?

Question 10

How many labeled binary trees are possible with 3 nodes?

56. Practice Answers

Answer 1

Given:

$$H = 3$$

Minimum nodes:

$$H + 1 = 3 + 1 = 4$$

Maximum nodes:

$$\begin{aligned} 2^{(H + 1)} - 1 \\ = 2^4 - 1 \\ = 16 - 1 \\ = 15 \end{aligned}$$

Answer:

minimum nodes = 4
maximum nodes = 15

Answer 2

Given:

$$H = 4$$

Minimum nodes:

$$H + 1 = 4 + 1 = 5$$

Maximum nodes:

$$\begin{aligned} 2^{(H + 1)} - 1 \\ = 2^5 - 1 \\ = 32 - 1 \\ = 31 \end{aligned}$$

Answer:

minimum nodes = 5
maximum nodes = 31

Answer 3

Given:

$$N = 6$$

Minimum height:

$$\begin{aligned} & \text{ceil}(\log_2(N + 1)) - 1 \\ &= \text{ceil}(\log_2(7)) - 1 \\ &= \text{ceil}(2.8) - 1 \\ &= 3 - 1 \\ &= 2 \end{aligned}$$

Maximum height:

$$N - 1 = 6 - 1 = 5$$

Answer:

$$\begin{aligned} \text{minimum height} &= 2 \\ \text{maximum height} &= 5 \end{aligned}$$

Answer 4

Given:

$$N = 15$$

Minimum height:

$$\begin{aligned} & \text{ceil}(\log_2(N + 1)) - 1 \\ &= \text{ceil}(\log_2(16)) - 1 \\ &= \text{ceil}(4) - 1 \\ &= 3 \end{aligned}$$

Maximum height:

$$N - 1 = 15 - 1 = 14$$

Answer:

minimum height = 3
maximum height = 14

Answer 5

Given:

$$\text{degree}(2) = 4$$

Formula:

$$\text{degree}(0) = \text{degree}(2) + 1$$

So:

$$\begin{aligned}\text{degree}(0) &= 4 + 1 \\ &= 5\end{aligned}$$

Answer:

5 leaf nodes

Answer 6

Given:

$$\text{degree}(0) = 9$$

Formula:

$$\text{degree}(0) = \text{degree}(2) + 1$$

So:

$$9 = \text{degree}(2) + 1$$

Therefore:

$$\text{degree}(2) = 8$$

Answer:

8 degree-2 nodes

Answer 7

For 3 nodes:

$$\text{Catalan}(3) = 5$$

Answer:

5 shapes

Answer 8

For 5 nodes:

$$\text{Catalan}(5) = 42$$

Answer:

42 shapes

Answer 9

Formula:

$$\text{Maximum-height binary trees} = 2^{(n - 1)}$$

Given:

$$n = 5$$

So:

$$2^{(5 - 1)} = 2^4 = 16$$

Answer:

16 maximum-height binary trees

Answer 10

Formula:

$$\text{Labeled binary trees} = \text{Catalan}(n) * n!$$

Given:

$$n = 3$$

So:

$$\begin{aligned}\text{Catalan}(3) &= 5 \\ 3! &= 6\end{aligned}$$

Therefore:

Labeled binary trees = $5 * 6 = 30$

Answer:

30 labeled binary trees

57. Final Beginner Checklist

Before moving to Phase 3, make sure you can answer these questions:

1. What is the height of a single-node tree?
2. Why does height start at 0 in this curriculum?
3. What is the minimum number of nodes for height H?
4. What is the maximum number of nodes for height H?
5. What is the maximum height for N nodes?
6. Why do we use ceiling in the minimum-height formula?
7. What is a degree 0 node?
8. What is a degree 1 node?
9. What is a degree 2 node?
10. What does $\text{degree}(0) = \text{degree}(2) + 1$ mean?
11. What does Catalan(n) count?
12. What is the difference between unlabeled and labeled binary trees?
13. Why do labeled binary trees use $n!$?
14. What does $2^{(n-1)}$ count?

If you can answer these, you understand Phase 2 well.

58. Final Phase 2 Memory Trick

Remember this:

Height-node formulas describe tree size limits.
Degree formulas describe leaf relationships.
Catalan numbers describe tree shape counts.
Factorial counts label arrangements.

So Phase 2 is mainly about four things:

height
nodes

degree counts
shape counts

Once these are clear, Phase 3 becomes easier because you will know the shapes before learning how to store them in memory.